

Algorithms

Dr. Mudassir Shabbir

LUMS University

April 1, 2026

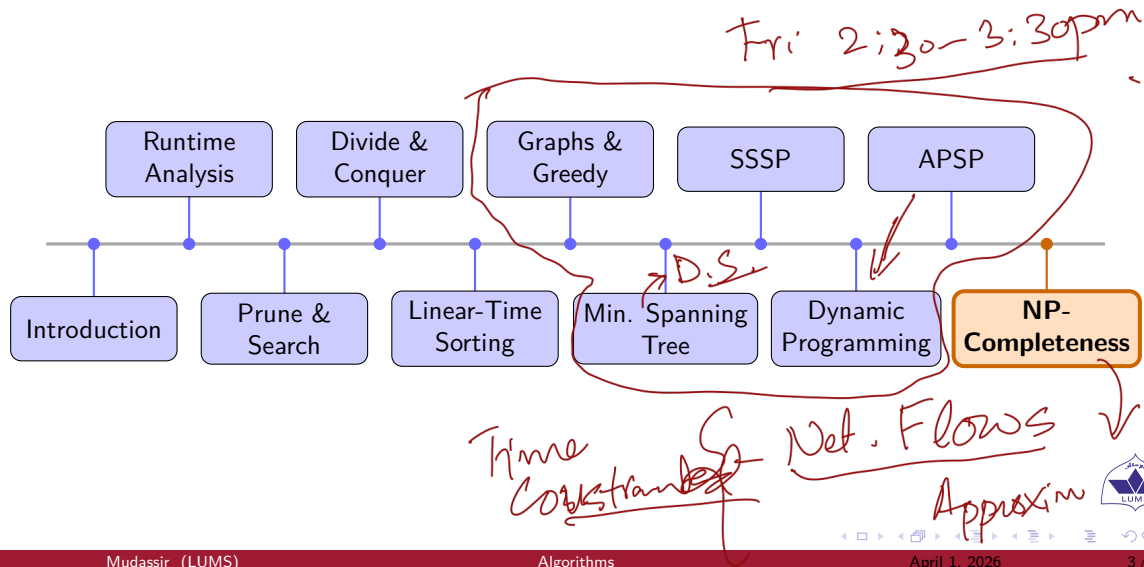


Announcements

- HW 5 is due on [Apr 05](#) at **11:59 AM**.
- **LQ 2** is on [Apr 05](#) at **6:30 PM**.



Recap & What's Next



NP-Completeness



Why Are We Here Now?

So far, much of the course has had the following flavor:

- define a computational problem,
- design a clever algorithm,
- prove it is correct,
- analyze its running time.

$\Omega(n \log n)$.

proved a lower bound on time complexity of all possible sorting algo. that are comparison based.

For many classic problems, that worked beautifully:

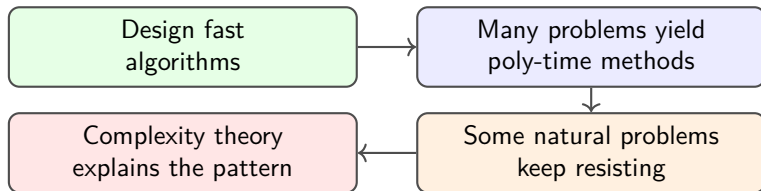
sorting, shortest paths, MST, dynamic programming, flows
($\Omega(n)$ $\Omega(V+E)$ optimal algo for MST $\Theta(?)$) Ap 13/15

But eventually we hit a different kind of question:

What if the problem itself seems to resist efficient algorithms?



A Chronological Shift in the Story



Early algorithm design asks:

How do we solve this problem efficiently?

NP-completeness asks the next historical question:

Should we even expect an efficient exact algorithm at all?

complexity theory gave a language for the difference between problems that seem tractable and problems that appear inherently hard.



Decision Problems vs Optimization Problems

class of problem = $\{S, P, SS, \dots\}$

There are two closely related kinds of computational problems:

Decision problem
answer is **Yes/No**

Optimization problem
find the **best** solution

May $\sum w_i$ Cheapest path / Longest path

- In complexity theory, we usually work with **decision problems**.
- Optimization versions are often the algorithmic form we actually care about in practice.

$\{0/1\}$ Yes/No T/F.

min. weight spanning tree.
 $\sum w_i$



Optimization Problems

An **optimization problem** asks for the **best** feasible solution.

Find the minimum tour cost



best value + actual solution



Examples:

- Find the shortest travelling salesman tour. ✓
- Find the largest clique in a graph. ✗
- Find the minimum number of colors needed to color a graph. ✓

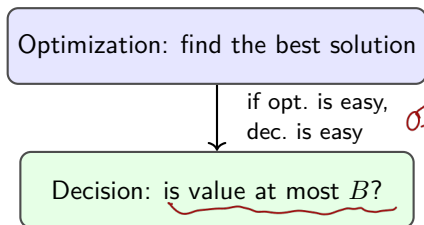


Why Complexity Theory Uses Decision Problems

Optimization and decision versions are often paired.

Example: TSP

- Optimization: find the minimum-cost tour.
- Decision: is there a tour with total cost at most B ?



find $S(A, B)$



Does there exist
a path from
 A to B
of total value
 $\leq X$?

So NP-completeness is usually stated for the **decision version** of a problem.



Travelling Salesman — Optimization Version

$G = (V, E, W)$, find a tour from vertex a to every node in V , and back to a that has min weight?

Example (4 cities, complete graph):

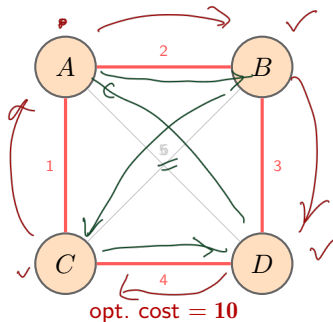
- There are 3 distinct tours.
- Optimal tour: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$
- **Minimum cost** = $2 + 3 + 4 + 1 = 10$

Decision Problem:

$\exists$ a tour of cost $\leq x$?

Path

Hamiltonian Cycle Problem: $G = (V, E)$, $\exists$ a cycle that visits every vertex (exactly once).

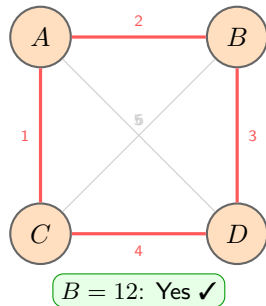


Travelling Salesman — Decision Version

Decision Input = $\{G, x\}$

Same 4-city example:

- $B = 12$: **Yes** — tour $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$
costs $10 \leq 12$ ✓
- $B = 9$: **No** — cheapest tour costs $10 > 9$



Maximum Clique — Optimization Version

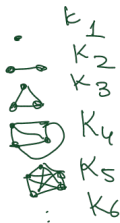
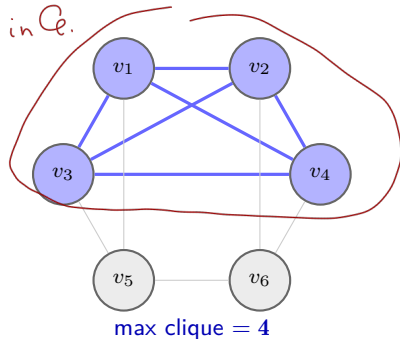
K_n as a subgraph.

Complete Graph: contains all possible edges.

Example (6 vertices):

- $\{v_1, v_2, v_3, v_4\}$ are all pairwise connected.
- No 5-vertex clique exists in this graph.
- **Maximum clique size = 4**

Given $G(V, E)$, find the maximum size clique in G .

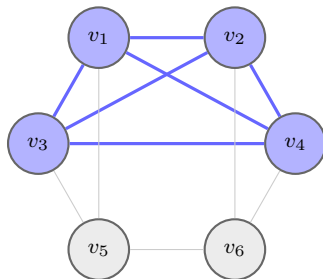


Maximum Clique — Decision Version

Same 6-vertex example:

- $k = 3$: **Yes** — $\{v_1, v_2, v_3\}$ ✓
- $k = 4$: **Yes** — $\{v_1, v_2, v_3, v_4\}$ ✓
- $k = 5$: **No** — no 5-clique exists

$\exists$ a clique on $\geq k$ vertices?



$k = 4$: Yes ✓



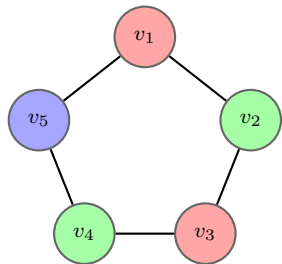
Graph Coloring — Optimization Version

Proper Coloring: coloring of V so that every adjacent pair gets different colors

Example (C_5 , the 5-cycle):

- $k = 2$: impossible (C_5 is an odd cycle).
- $k = 3$: valid coloring exists (shown right).
- $\chi(C_5) = 3$

problem find min. of colors required for proper coloring of G .



$$\chi(C_5) = 3$$

Decision: $\exists$ a proper coloring on $\leq k$ colors.

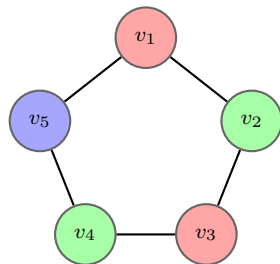
then $\exists k$ needs k colors. every other graph needs fewer.



Graph Coloring — Decision Version

Same C_5 example:

- $k = 3$: **Yes** ✓
- $k = 2$: **No** — odd cycle requires ≥ 3 colors



$k = 3$: Yes ✓

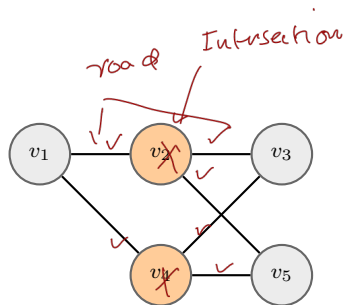


Vertex Cover — Optimization Version

Example (5 vertices, 6 edges):

- $\{v_2, v_4\}$ (highlighted) cover all 6 edges.
- No single vertex covers all edges.
- **Min cover size = 2**

Problem: find min. number
of vertices needed to
Cover all edges.



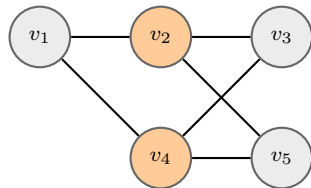
min cover = $\{v_2, v_4\}$, size 2

Decision:
Is a v.c. of
size $\leq k$?

Vertex Cover — Decision Version

Same example:

- $k = 2$: **Yes** — $\{v_2, v_4\}$ covers every edge ✓
- $k = 1$: **No** — no single vertex touches all edges



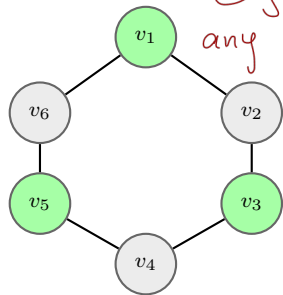
$k = 2$: Yes ✓

Independent Set — Optimization Version

IS Ind. Set. $\subseteq V$
so that there are no edges b/w any pair.

Example (C_6 , the 6-cycle):

- $\{v_1, v_3, v_5\}$ (highlighted): no two are adjacent.
- Adding any fourth vertex creates a conflict.
- **Max independent set:** size 3



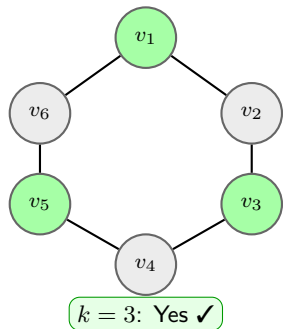
max IS size = 3

Opt. find max. I.S.
Decision. $\exists$ a I.S. of size $\geq k$.

Independent Set — Decision Version

Same C_6 example:

- $k = 3$: **Yes** — $\{v_1, v_3, v_5\}$ ✓
- $k = 4$: **No** — max IS has size 3



0-1 Knapsack — Optimization Version

Opt.

subset of items of max. value
that (can fit in your knapsack.)

Example ($W = 7$):

Item	Weight	Value	Take?
A	3	8	✓
B	4	6	✓
C	2	5	—
D	5	9	—

Decision:

∃ a subset

of value $\geq k$

that ⊗

$A + B$: weight = $3 + 4 = 7 \leq W$; value = $8 + 6 = 14$ (**optimal**).



0-1 Knapsack — Decision Version

Same example ($W = 7$):

- $V = 14$: **Yes** — select $\{A, B\}$, weight = 7, value = $14 \geq 14$ ✓
- $V = 15$: **No** — maximum achievable value is 14



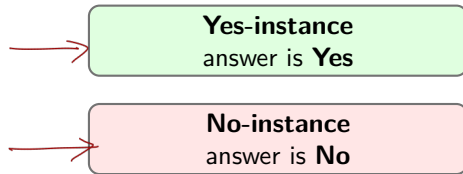
Instances of a Decision Problem

P

An **instance** of a decision problem is a specific, concrete input.

Clique: the decision problem asks “does graph G contain a clique of size $\geq k$?”

An **instance** is a particular pair (G, k) , e.g. the 6-vertex graph from before with $k = 4$.



$\{1, 5, 2\}$

Every instance of a decision problem is either a yes-instance or a no-instance.



The Problems That Forced the Question

Each of the following is easy to **state** and easy to **verify**, yet no polynomial-time algorithm has ever been found:

Travelling Salesman

Scheduling

Graph 3-Coloring

Subset Sum

NP-completeness is the framework that explains why these problems are **believed to be hard** - and what to do about it.



What Does It Mean to Verify a Solution?

When we say **verify**, we do *not* mean:

solve the whole problem from scratch

We mean something weaker and faster:

Verification

Given an input instance and a proposed solution, check whether that proposed solution really proves that the answer is **yes**.

So verification has two inputs:

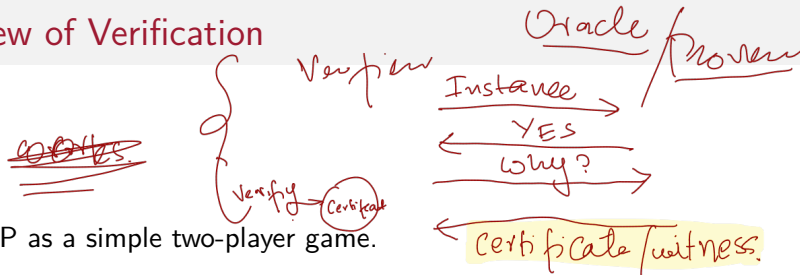
- the original instance,
- a candidate witness supplied by someone else.

The verifier's job is only to say:

accepted or **rejected**

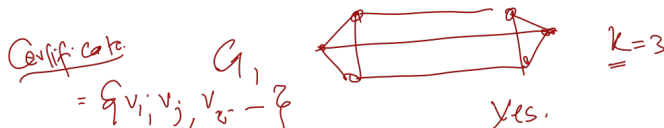


A Two-Player View of Verification



It helps to think of NP as a simple two-player game.

- The prover may be all-powerful; we do not care how the certificate was found.
- The verifier must be efficient and skeptical.
- NP asks whether **yes-instances** have certificates that can be checked in polynomial time.



What Is a Certificate?

A **certificate** is the extra information given by the prover that is supposed to convince the verifier that the answer is **yes**.

You can think of a certificate as:

- a witness,
- a proof sketch,
- a candidate solution that can be checked quickly.



When Does Verification Count as Efficient?

Certificate has size $O(n^k)$ k constant

For NP, we require the verifier to run in polynomial time in the size of the input.

That means:

- the certificate must be of reasonable size,
- the checking procedure must finish in time like $O(n^2)$ or $O(n^3)$,
- the verifier cannot secretly do exponential search while "checking".

Verification is only required for yes-instances. If the true answer is **no**, there may be no short certificate at all.

This is why NP is about efficiently verifiable **yes-proofs**.



A First Big Distinction: Solve vs Verify

Sometimes **finding** a solution seems hard, even when **checking** a proposed solution is easy.

Example: Hamiltonian Cycle

- Finding such a cycle from scratch may be difficult.
- But if someone hands you an ordered list of vertices, you can quickly check:
 - does every vertex appear exactly once?
 - does each consecutive pair have an edge?
 - does the last vertex connect back to the first?

So a certificate for a **yes** instance is just the proposed cycle itself.

This gap between **searching** and **verifying** is the doorway into NP.



Class P

P is the class of decision problems that can be solved in polynomial time.

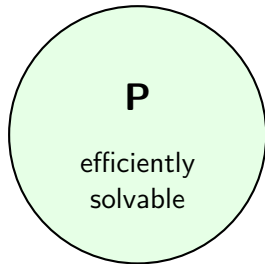
Polynomial time means running time like

$$O(n), O(n^2), O(n^3), O(n^{10})$$

for some polynomial in the input size.

Interpretation: P is our formal notion of efficiently solvable problems.

Examples: shortest path, MST, max flow, primality.



Class NP

Non deterministic Polynomial. $C, k = n/2$
 $\geq k$
 $O(n^{n/2 - \epsilon}) \approx \binom{n}{n/2}$

NP is the class of decision problems for which a **yes-certificate** can be verified in polynomial time.

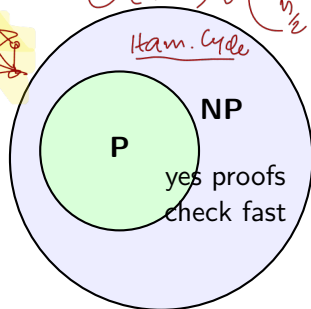
If the answer is **yes**, there is a short witness we can check quickly.

Examples of certificates:

Hamiltonian cycle

- chosen subset
- proposed coloring
- proposed TSP tour

Every problem in P is also in NP.



Yes-instances
can verified
in $O(n^k)$, k constant.
Clique $\in$ NP.

The Question Behind the Entire Topic

We know:

$$P \subseteq NP$$

The great open question is:

Is $P = NP$?

Informally:

Does every problem whose solutions can be verified quickly also have a fast algorithm to find those solutions?

Most computer scientists believe the answer is **no**.

